AutoBayes against brms: where the speed comes from

Benchmark run of 2026-10-25, prepared for Andrew

2026-10-25

The answer. Fitting each model from scratch (compiling and sampling), brms took 0.8 to 18.1 times as long as AutoBayes (median 6.5). Its sampler alone (warm-up and sampling of every chain, compilation excluded) took 0.6 to 13.2 times as long (median 3.8). Splitting that ratio into its four factors (Section 4), the largest was the gradient factor (for AutoBayes) on 15 models, the overhead factor (against AutoBayes) on 2 models, the gradient factor (against AutoBayes) on 1 model. brms's sampler was the faster on B8, B12.

1 What was run

Each of the 18 models was fitted by both libraries with the same formula, family and explicit priors, on the same simulated data (`examples/brms_parity/benchmark_data.py`, fixed seeds), with the same sampler settings: 4 chains in parallel, 1000 warm-up and 1000 kept draws each, target acceptance 0.9.

- **brms** 2.23.0 with rstan 2.32.7 (R 4.5.2), 4 cores; compiled with `brm(chains = 0)` and sampled with `update(recompile = FALSE)`.
- **AutoBayes** (commit `ce04dd3`) with NumPyro 0.21.0, JAX 0.10.2 and PyMC 5.28.5 on the CPU (macOS-26.6.1-arm64-arm-64bit, 10 cores), `ab.fit` unchanged and instrumented from outside (`examples/brms_parity/README.md`).

2 Time to a posterior, by stage

	model	rows	codegen	compile	exec	around	total
B1	Bernoulli, `(1 + x | g)`	2,000	0.09	31.1	4.5	1.2	36.8
B2	Gaussian, `(1 + x | g)`	20,000	0.12	29.2	74.9	1.8	105.9
B3	cumulative, five categories	1,500	0.07	26.1	32.4	0.44	58.9
B4	zero-inflated Poisson	2,000	0.09	25.5	14.6	0.49	40.6
B5	Weibull	2,000	0.08	26.3	5.9	0.51	32.7
B6	zero-inflated neg. binomial	2,000	0.08	26.2	15.9	0.53	42.7
B7	sratio, five categories	1,500	0.07	25.9	17.1	0.47	43.5
B8	Gaussian, horseshoe on 20	400	0.09	25.5	1.7	0.73	27.9
B9	beta-binomial, trials	2,000	0.08	24.7	23.7	0.65	49.1
B10	Gaussian, `(1 | mm(g1, g2))`	1,000	0.08	24.7	2.3	0.58	27.6
B11	mixture of two Gaussians	1,000	0.10	27.0	4.0	0.51	31.5
B12	exact GP, `gp(x)`	200	0.07	27.6	468.8	1.2	497.6
B13	AR(1), `ar(t, g)`	2,000	0.10	25.0	2.1	1.2	28.3
B14	non-linear, `a * exp(b * x)`	1,000	0.11	25.2	5.7	0.66	31.6
B15	`mvbind(y1, y2)`, rescor	1,000	0.17	29.7	13.1	0.88	43.8
B16	monotonic `mo(edu)`	1,500	0.12	28.2	6.5	1.1	35.7
B17	Weibull, `cens()`	2,000	0.10	28.7	5.6	1.3	35.6
B18	distributional, `sigma ~ z`	2,000	0.11	26.9	8.5	0.72	36.2

Table 1: brms's stages, seconds. *compile*: `brm(chains = 0)`, code generation, C++ compilation and loading; *codegen*: the code generation alone, run again afterwards; *exec*: the slowest chain's warm-up and sampling as Stan reports them; *around*: the rest of `update()` (forking the chains, brms's data preparation and its renaming of the fit).

	build	pre	compile	JAX	exec	post	total	faster by	
								total	exec
B1	2.1	0.33	1.2	1.2	2.5	0.83	7.0	5.27	1.78
B2	10.4	0.61	1.3	1.3	17.4	0.86	30.5	3.47	4.31
B3	0.10	0.38	1.0	1.0	4.9	0.29	6.7	8.82	6.64
B4	0.12	0.48	1.1	1.1	4.1	0.22	6.0	6.82	3.61
B5	0.12	0.28	1.1	1.1	4.3	0.14	6.0	5.48	1.37
B6	0.32	0.53	1.4	1.4	12.7	0.15	15.2	2.81	1.25
B7	0.09	0.32	1.0	1.0	5.6	0.13	7.1	6.12	3.08
B8	0.09	0.48	1.2	1.2	3.0	0.32	5.1	5.46	0.56
B9	0.13	0.29	1.6	1.6	20.4	0.15	22.6	2.17	1.16
B10	0.09	0.24	1.0	1.00	0.47	0.12	1.9	14.38	4.86
B11	0.04	0.75	1.1	1.1	0.98	0.19	3.1	10.12	4.05
B12	0.05	0.28	1.1	1.1	617.9	0.50	619.8	0.80	0.76
B13	0.09	0.23	0.96	0.95	0.22	0.07	1.6	18.11	9.79
B14	0.05	0.23	0.99	0.99	0.83	0.11	2.2	14.35	6.89
B15	0.16	0.62	1.6	1.6	1.00	0.23	3.6	12.17	13.16
B16	0.49	0.43	1.2	1.2	0.98	0.26	3.4	10.64	6.59
B17	0.14	0.48	1.2	1.1	4.9	0.15	6.8	5.21	1.13
B18	0.63	0.25	1.1	1.0	1.6	0.15	3.7	9.90	5.43

Table 2: AutoBayes's stages, seconds. *build*: the model; *pre*: the fit up to NumPyro (PyTensor's graph turned into JAX, initial points); *compile*: the NumPyro run up to the end of its largest XLA compilation, with *JAX* the part of it JAX's monitoring reports as tracing, lowering and compiling; *exec*: the rest of the run, every chain's warm-up and sampling; *post*: the draws moved to the constrained scale and the InferenceData. The last two columns are brms's time over AutoBayes's, end to end and for the samplers' execution.

3 The samplers' work

	brms				AutoBayes				
	warm-up	sampling	depth	step	warm-up	sampling	depth	step	ratio
B1	98,770	61,840	4.01	0.247	95,315	60,000	4.00	0.260	1.03
B2	221,936	124,000	5.00	0.135	217,750	124,000	5.00	0.129	1.01
B3	87,144	61,720	3.98	0.240	84,526	61,272	4.02	0.227	1.02
B4	113,190	90,016	4.29	0.179	111,279	68,112	4.12	0.206	1.13
B5	98,806	68,624	4.06	0.186	102,278	73,592	4.20	0.176	0.95
B6	92,569	60,064	3.99	0.269	89,590	59,984	4.00	0.263	1.02
B7	73,807	59,216	3.87	0.300	70,200	58,360	3.94	0.301	1.03
B8	530,028	499,873	6.94	0.034	537,367	492,265	6.94	0.035	1.00
B9	101,875	78,208	4.14	0.167	100,612	77,552	4.26	0.165	1.01
B10	105,127	78,304	4.11	0.189	99,732	77,864	4.27	0.196	1.03
B11	34,102	29,820	2.93	0.495	33,580	28,948	3.01	0.486	1.02
B12	1,016,378	1,005,531	7.96	0.019	989,870	1,019,054	8.00	0.018	1.01
B13	23,095	22,850	2.58	0.663	24,532	22,270	2.64	0.685	0.98
B14	172,782	109,968	4.34	0.119	182,541	113,000	4.69	0.111	0.96
B15	127,644	85,680	4.20	0.205	122,077	81,024	4.33	0.202	1.05
B16	143,761	80,632	4.14	0.177	136,401	88,744	4.43	0.169	1.00
B17	94,509	66,336	4.02	0.195	87,164	61,760	4.03	0.227	1.08
B18	114,805	100,160	4.39	0.145	110,730	92,320	4.47	0.150	1.06

Table 3: Sampler work: leapfrog steps summed over the chains, for warm-up and for sampling; the mean tree depth while sampling and the final step size, averaged over the chains. The last column is brms's steps over AutoBayes's.

		brms				AutoBayes				
	coords	host	direct	null	used	host	null	loop	compile	ratio
B1	105	109.0	103.0	5.0	98.0	109.5	7.1	52.7	0.16	1.86
B2	407	809.2	805.1	5.0	800.1	250.4	7.7	110.6	0.16	7.24
B3	36	770.1	761.0	5.0	756.0	167.8	7.3	102.1	0.15	7.40
B4	44	253.9	242.0	5.0	237.0	122.4	7.2	73.6	0.13	3.22
B5	44	129.0	123.0	5.0	118.0	140.1	6.9	77.9	0.16	1.51
B6	45	381.0	376.0	5.0	371.0	282.8	7.0	290.4	0.28	1.28
B7	36	473.0	467.1	5.0	462.1	213.2	6.9	145.2	0.13	3.18
B8	44	16.0	11.0	5.0	6.0	87.0	7.1	13.9	0.19	0.43
B9	44	474.6	476.8	5.0	471.8	362.1	7.0	285.0	0.28	1.66
B10	34	55.1	50.1	5.0	45.1	52.0	7.3	4.6	0.10	9.73
B11	7	221.0	212.0	5.0	206.9	92.2	7.2	32.8	0.20	6.31
B12	204	922.0	908.1	5.0	903.1	1,287	7.0	952.0	0.20	0.95
B13	4	158.1	150.9	5.0	145.9	63.3	6.8	12.1	0.10	12.09
B14	24	78.0	73.9	5.0	68.9	53.0	7.2	5.9	0.09	11.65
B15	69	227.0	219.1	5.0	214.1	81.8	8.4	12.1	0.19	17.74
B16	38	109.0	99.9	5.0	94.9	72.7	7.7	11.6	0.16	8.20
B17	44	139.0	134.0	5.0	129.0	180.4	7.0	117.0	0.18	1.10
B18	45	145.0	143.1	5.0	138.0	80.8	7.0	22.5	0.10	6.14

Table 4: The cost of one log density and gradient, microseconds, at a posterior draw on the unconstrained scale (the number of coordinates first). brms: `rstan::grad_log_prob` (*host*), the model instance's method called directly (*direct*), a call that does no work (*null*), and *direct* less *null*, the estimate used below. AutoBayes: the jitted potential and gradient called from Python (*host*), a jitted call that does no work (*null*), and the cost per evaluation inside one compiled loop (*loop*), the estimate used below; *compile* is seconds to trace, lower and compile it. Medians of 2,000 calls after a first one.

4 Where the sampling time goes

For each side, T is the samplers' execution time (warm-up and sampling of every chain, compilation excluded), S the leapfrog steps summed over the chains, g the cost of one log density and gradient inside the sampler (Table 4) and C the CPU seconds spent over T. With $O = C/(Sg)$, the CPU cost of one step counted in gradients (1 if a step costs exactly a gradient), and $U = C/T$, the cores kept busy, $T = SgO/U$ holds exactly, so brms's T over AutoBayes's is the product

$$\frac{T_b}{T_a} = \underbrace{\frac{S_b}{S_a}}_{\text{steps}} \cdot \underbrace{\frac{g_b}{g_a}}_{\text{gradient}} \cdot \underbrace{\frac{O_b}{O_a}}_{\text{overhead}} \cdot \underbrace{\frac{U_a}{U_b}}_{\text{parallelism}} .$$

A worked example, B11 (the median sampler speed-up). brms took $T = 3.97$ s and 63,922 leapfrog steps over all chains; at $g = 206.9\,\mu$s a gradient those steps alone are $Sg = 13.23$ CPU seconds. Its chains used $C = 15.93$ CPU seconds, so each step cost $O = C/(Sg) = 1.20$ gradients and $U = C/T = 4.01$ cores were busy on average. AutoBayes took $T = 0.98$ s and 62,528 leapfrog steps over all chains; at $g = 32.8\,\mu$s a gradient those steps alone are $Sg = 2.05$ CPU seconds. Its chains used $C = 4.48$ CPU seconds, so each step cost $O = C/(Sg) = 2.18$ gradients and $U = C/T = 4.56$ cores were busy on average. Then $3.97/0.98 = 4.05$, which is 1.02 (steps) times 6.31 (gradient) times 0.55 (overhead) times 1.14 (parallelism).

	T (s)			factor				O		U	
	brms	AutoBayes	ratio	steps	gradient	overhead	parallel	brms	AutoBayes	brms	AutoBayes
B1	4.5	2.5	1.78	1.03	1.86	0.92	1.01	1.12	1.22	3.90	3.94
B2	74.9	17.4	4.31	1.01	7.24	0.35	1.66	1.03	2.92	3.82	6.34
B3	32.4	4.9	6.64	1.02	7.40	0.85	1.04	1.08	1.27	3.75	3.88
B4	14.6	4.1	3.61	1.13	3.22	1.01	0.98	1.11	1.09	3.65	3.56
B5	5.9	4.3	1.37	0.95	1.51	0.99	0.96	1.11	1.12	3.74	3.57
B6	15.9	12.7	1.25	1.02	1.28	0.97	0.99	1.08	1.12	3.86	3.83
B7	17.1	5.6	3.08	1.03	3.18	0.94	1.00	1.09	1.16	3.91	3.91
B8	1.7	3.0	0.56	1.00	0.43	1.44	0.91	1.17	0.81	4.25	3.85
B9	23.7	20.4	1.16	1.01	1.66	0.41	1.69	1.07	2.61	3.84	6.48
B10	2.3	0.47	4.86	1.03	9.73	0.54	0.89	1.12	2.08	4.10	3.66
B11	4.0	0.98	4.05	1.02	6.31	0.55	1.14	1.20	2.18	4.01	4.56
B12	468.8	617.9	0.76	1.01	0.95	0.49	1.61	1.00	2.02	3.89	6.25
B13	2.1	0.22	9.79	0.98	12.09	0.89	0.93	1.32	1.48	4.17	3.88
B14	5.7	0.83	6.89	0.96	11.65	0.65	0.95	1.12	1.72	3.82	3.64
B15	13.1	1.00	13.16	1.05	17.74	0.73	0.96	1.09	1.49	3.79	3.65
B16	6.5	0.98	6.59	1.00	8.20	0.79	1.02	1.10	1.40	3.62	3.71
B17	5.6	4.9	1.13	1.08	1.10	1.02	0.93	1.11	1.08	4.13	3.84
B18	8.5	1.6	5.43	1.06	6.14	0.88	0.95	1.07	1.22	3.73	3.53

Table 5: The four factors of brms's sampler time over AutoBayes's, and each side's O and U. A factor above 1 favours AutoBayes.

Model by model, the largest factor:

- **B1** (Bernoulli, `(1 + x | g)`): sampler 1.78 times faster in AutoBayes; largest factor gradient (1.86, cheaper gradients).
- **B2** (Gaussian, `(1 + x | g)`): sampler 4.31 times faster in AutoBayes; largest factor gradient (7.24, cheaper gradients).
- **B3** (cumulative, five categories): sampler 6.64 times faster in AutoBayes; largest factor gradient (7.40, cheaper gradients).
- **B4** (zero-inflated Poisson): sampler 3.61 times faster in AutoBayes; largest factor gradient (3.22, cheaper gradients).
- **B5** (Weibull): sampler 1.37 times faster in AutoBayes; largest factor gradient (1.51, cheaper gradients).
- **B6** (zero-inflated neg. binomial): sampler 1.25 times faster in AutoBayes; largest factor gradient (1.28, cheaper gradients).
- **B7** (sratio, five categories): sampler 3.08 times faster in AutoBayes; largest factor gradient (3.18, cheaper gradients).
- **B8** (Gaussian, horseshoe on 20): sampler 0.56 times faster in AutoBayes; largest factor gradient (0.43, against AutoBayes).
- **B9** (beta-binomial, trials): sampler 1.16 times faster in AutoBayes; largest factor overhead (0.41, against AutoBayes).
- **B10** (Gaussian, `(1 | mm(g1, g2))`): sampler 4.86 times faster in AutoBayes; largest factor gradient (9.73, cheaper gradients).
- **B11** (mixture of two Gaussians): sampler 4.05 times faster in AutoBayes; largest factor gradient (6.31, cheaper gradients).
- **B12** (exact GP, `gp(x)`): sampler 0.76 times faster in AutoBayes; largest factor overhead (0.49, against AutoBayes).
- **B13** (AR(1), `ar(t, g)`): sampler 9.79 times faster in AutoBayes; largest factor gradient (12.09,

cheaper gradients).

- **B14** (non-linear, `a * exp(b * x)`): sampler 6.89 times faster in AutoBayes; largest factor gradient (11.65, cheaper gradients).
- **B15** (`mvbind(y1, y2)`, rescor): sampler 13.16 times faster in AutoBayes; largest factor gradient (17.74, cheaper gradients).
- **B16** (monotonic `mo(edu)`): sampler 6.59 times faster in AutoBayes; largest factor gradient (8.20, cheaper gradients).
- **B17** (Weibull, `cens()`): sampler 1.13 times faster in AutoBayes; largest factor gradient (1.10, cheaper gradients).
- **B18** (distributional, `sigma ~ z`): sampler 5.43 times faster in AutoBayes; largest factor gradient (6.14, cheaper gradients).

5 Is the faster sampler sampling as well?

	min bulk ESS		min tail ESS		bulk ESS/s		divergent		max $\lvert z \rvert$
	AutoBayes	brms	AutoBayes	brms	AutoBayes	brms	AutoBayes	brms	
B1	1,264	1,017	1,843	1,638	500	226	0	0	1.88
B2	125	144	298	459	7	2	0	0	1.22
B3	530	558	1,062	1,076	109	17	0	0	0.41
B4	386	414	726	951	95	28	0	0	1.46
B5	201	287	296	541	47	49	0	0	1.02
B6	1,079	1,245	1,874	2,019	85	78	0	0	0.70
B7	1,087	1,195	1,418	1,717	196	70	0	0	1.86
B8	1,916	2,257	1,846	1,313	637	1,342	9	6	2.71
B9	185	264	465	499	9	11	0	0	1.43
B10	533	499	929	835	1,143	220	0	0	0.82
B11	3,325	3,131	2,974	2,922	3,390	788	0	0	1.49
B12	1,214	1,410	1,709	1,799	2	3	2	4	0.57
B13	4,174	4,536	2,801	2,842	19,306	2,143	0	0	0.78
B14	290	269	446	379	351	47	0	0	0.51
B15	806	1,022	1,499	1,754	808	78	0	0	1.11
B16	440	400	675	893	448	62	0	0	1.36
B17	355	309	625	660	72	56	0	0	0.69
B18	192	200	377	277	122	23	0	0	1.30

Table 6: Sampling efficiency and agreement: the smallest bulk and tail ESS over the parameters both libraries report (the same ArviZ estimators for both), the smallest bulk ESS per second of the samplers' execution T, the divergent transitions after warm-up, and the largest $|z|$, the difference of the posterior means over their combined Monte Carlo standard error.

6 Post-processing

	posterior_epred	log_lik	loo	waic	bayes_R2	hypothesis	cond._effects
B1	0.12 / 0.35	0.53 / 0.71	2.16 / 3.23	0.64 / 1.79	0.16 / 0.63	0.00 / 0.00	0.02 / 0.15
B2	0.68 / 1.20	2.49 / 5.27	18.97 / 32.10	3.84 / 17.60	0.89 / 3.80	0.00 / 0.00	0.02 / 0.17
B3	1.77 / 1.65	0.58 / 1.23	1.77 / 3.67	0.66 / 2.46	1.94 / 2.35	*ref.* / 0.00	*ref.* / 0.27
B4	0.13 / 0.35	0.35 / 1.39	2.00 / 4.09	0.47 / 2.92	0.17 / 0.63	0.00 / 0.00	0.02 / 0.14
B5	0.12 / 0.38	0.19 / 1.06	1.87 / 3.50	0.29 / 2.11	0.16 / 0.68	0.00 / 0.00	0.02 / 0.15
B6	0.14 / 0.36	1.00 / 1.80	2.65 / 4.26	1.11 / 3.14	0.18 / 0.66	0.00 / 0.00	0.02 / 0.15
B7	1.31 / 9.78	1.22 / 1.95	2.35 / 4.40	1.28 / 3.03	1.44 / 10.09	*ref.* / 0.00	*ref.* / 0.76
B8	0.04 / 0.06	0.03 / 0.10	0.33 / 0.58	0.05 / 0.35	0.05 / 0.13	0.00 / 0.00	0.03 / 0.20
B9	0.16 / 0.41	1.48 / *ref.*	3.13 / *ref.*	1.59 / *ref.*	0.19 / 0.69	0.00 / 0.00	*ref.* / 0.15
B10	0.05 / 0.10	0.08 / 0.21	0.82 / 1.61	0.14 / 0.99	0.07 / 0.21	0.00 / 0.00	0.02 / 0.15
B11	0.05 / 0.12	0.29 / 0.60	1.00 / 2.18	0.32 / 1.40	0.06 / 0.33	*ref.* / 0.00	*ref.* / 0.15
B12	2.66 / 3.58	2.00 / 3.71	2.53 / 3.97	2.55 / 3.79	1.77 / 3.57	0.00 / 0.00	*ref.* / 13.32
B13	0.17 / 0.79	0.23 / 1.06	1.90 / 3.74	0.33 / 2.23	0.20 / 1.13	0.00 / 0.00	*ref.* / 0.15
B14	0.06 / 0.32	0.09 / 0.43	0.86 / 1.62	0.15 / 1.20	0.08 / 0.44	*ref.* / 0.00	*ref.* / 0.16
B15	0.10 / 0.55	*ref.* / 62.95	*ref.* / 64.63	*ref.* / 64.96	*ref.* / 0.71	*ref.* / 0.00	*ref.* / 0.47
B16	0.09 / 0.34	0.13 / 0.68	1.30 / 2.43	0.21 / 1.61	0.11 / 0.66	0.00 / 0.00	*ref.* / 0.18
B17	0.13 / 0.33	0.51 / 0.89	2.21 / 3.41	0.58 / 2.26	0.17 / 0.67	0.00 / 0.00	*ref.* / 0.15
B18	0.09 / 0.41	0.23 / 0.69	1.96 / 3.32	0.33 / 2.17	0.13 / 0.80	0.00 / 0.00	*ref.* / 0.18

Table 7: Post-processing, seconds, AutoBayes / brms: the median of three calls after a first one. *ref.*: the function refused the model (the message is in the JSON).

7 Against the previous run

	brms		AutoBayes	
	before	now	before	now
B1	42.0	36.8	5.3	7.0
B2	122.5	105.9	29.6	30.5
B3	60.4	58.9	7.1	6.7
B4	41.2	40.6	5.9	6.0
B5	33.5	32.7	6.0	6.0
B6	43.0	42.7	14.7	15.2
B7	43.6	43.5	7.1	7.1
B8	29.6	27.9	5.3	5.1
B9	52.1	49.1	22.3	22.6
B10	30.0	27.6	2.4	1.9

Table 8: End-to-end seconds in the previous run and in this one, for the models both have.

8 What these numbers do not show

- **One run each**, on one machine whose load is not controlled: a single number may be off by 10–30%.
- **Step counts differ for reasons beyond the gradient.** Stan and NumPyro both run NUTS with a diagonal metric and dual-averaging step-size adaptation, but their warm-up schedules,

windows and step-size searches are not the same, and they start from different points, so they end on different step sizes and tree depths. The steps factor measures that difference; it says nothing about either library's gradient. Neither count includes the gradients spent searching for an initial step size.

- **The gradient costs are estimates.** The host-call medians include a call overhead the samplers do not pay (for a jitted JAX function on the CPU, tens of microseconds on a busy machine); the factors use AutoBayes's cost inside a compiled loop and rstan's direct call less a call that does no work. brms's point is a draw of a short extra chain (brms renames its fit's parameters, so the fit's draws cannot be mapped back to Stan's coordinates); AutoBayes's is the last draw of its fit. For these models the cost hardly depends on the point.
- **CPU time and parallelism.** AutoBayes's C is the whole process's CPU time over the run's execution (XLA's threads included); brms's is the CPU time of the forked chain processes over `update()`, which includes each child's small R-side setup. U above the number of chains means a chain used more than one core.
- **Compilation is measured from the timeline.** AutoBayes's sampler compilation is the time from the start of the NumPyro run to the end of its largest XLA compilation; the part JAX reports as tracing, lowering and compiling is shown beside it. brms's codegen is a warm re-run of `stancode()` and `standata()`.
- **Repeated fits.** brms compiles once per model and can refit new data without recompiling; for many fits of one model only the samplers' time matters.
- **Other setups.** rstan only (cmdstanr is not installed), no within-chain threading, the CPU only.

9 Reproducing

```
python examples/brms_parity/benchmark_data.py DATA
Rscript examples/brms_parity/benchmark_brms.R DATA OUT
python examples/brms_parity/benchmark_autobayes.py DATA OUT
python examples/brms_parity/compare.py OUT
python examples/brms_parity/report.py OUT/comparison-DATE.json --pdf
```